

Recuperatorio 2do Parcial – 2020s2 - Estructuras de Datos – UNQ

Aclaraciones:

- Esta evaluación es a libro abierto. Se pueden usar todas las funciones y tipos de datos vistos en la práctica y en la teórica, salvo que el enunciado indique lo contrario.
- Se recomienda leer el parcial completo antes de comenzar a resolverlo.

Ronda de mate

Una empresa de simulación desea incorporar la capacidad de que se formen rondas de mate en sus simulaciones. Para ello quieren contratar a un/a programador/a residente en la República Argentina. Tienen diseñada la interfaz asociada a una ronda de mate, la representación deseada y los costos máximos de cada función. La especificación de la interfaz es la siguiente.

- `nuevaRondaMate :: Int -> [Persona] -> RondaMate`
Propósito: Dados un número n y una lista de personas, construye una nueva ronda de mate con las personas de la lista y una cantidad máxima de n mates.
Precondiciones: La lista no está vacía y el número es mayor que 0.
Eficiencia: $O(N)$
- `cebar :: RondaMate -> RondaMate`
Propósito: Dada una ronda de mate, computa la ronda de mate resultante de que se bebe un mate. El cebado consiste en que se sirva agua al mate y se pase a la siguiente persona en la ronda.
Precondiciones: Queda agua para un mate.
Eficiencia: $O(\log N)$
- `saltear :: Int -> RondaMate -> (RondaMate, [Persona])`
Propósito: Dadas una ronda de mate y un número n , computa la ronda de mate resultante de que se bebe un mate a la persona que tiene n personas delante en la ronda. Las personas que estaban delante no reciben mate en esta vuelta, es decir, la ronda continuará a partir del siguiente al que recibió el mate.
Precondiciones: Queda agua para un mate.
Eficiencia: $O(N)$
- `quitar :: Persona -> RondaMate -> Int`
Propósito: Dadas una persona y una ronda de mates, computa la ronda de mate resultante de quitar a la persona.
Nota: Cuando se quita una persona de la ronda, se debe eliminar de todas las estructuras internas, pero los mates que tomó deben ser considerados para el cómputo de los mates restantes.
Eficiencia: $O(N \times \log N)$
- `quienMenosTomó :: RondaMate -> (Persona, Int)`
Propósito: Dada una ronda de mate, computa el par compuesto por la persona que menos mates tomó junto a la cantidad de mates que tomó.
Eficiencia: $O(N)$
- `quienTieneElMate :: RondaMate -> Persona`
Propósito: Dada una ronda de mate, computa la persona que tiene el mate actualmente.
Precondiciones: Se cebó al menos un mate.
Eficiencia: $O(N)$
- `matesRestantes :: RondaMate -> Int`
Propósito: Dada una ronda de mates, computa la cantidad de mates restantes.
Eficiencia: $O(N)$
- `cantidadDePersonas :: RondaMate -> Int`
Propósito: Dada una ronda de mates, computa la cantidad de personas que la integran.
Eficiencia: $O(N)$

Además, quieren hacer uso de un TAD auxiliar Ronda, cuya interfaz es la siguiente.

■ `nuevaRonda :: [a] -> Ronda a`

Propósito: Dada una lista de elementos, construye una nueva ronda con los elementos de la lista.

Precondiciones: La lista no está vacía.

Eficiencia: $O(N)$

■ `siguiente :: Ronda a -> (a, Ronda a)`

Propósito: Dada una ronda de elementos, computa el par con el próximo elemento en la ronda, y la ronda resultante de avanzar un elemento.

Eficiencia: $O(1)$

■ `tamaño :: Ronda a -> Int`

Propósito: Dada una ronda de elementos, computa la cantidad de elementos que hay en ella.

Eficiencia: $O(N)$

■ `borrar :: a -> Ronda a -> Ronda a`

Propósito: Dado un elemento y una ronda de elementos, computa la ronda resultante de borrar al elemento de la misma.

Precondiciones: El elemento está en la ronda.

Eficiencia: $O(N)$

Por último, se dispone del TAD **Persona**, para el cual no hay interfaz definida pero se sabe que es comparable por igualdad (Eq).

Ejercicios

1. Implemente la siguiente función como usuario/a del TAD **RondaMate**, estableciendo y justificando su eficiencia y la de las funciones auxiliares.

a) `personasOrdenadas :: RondaMate -> [Persona]`

Propósito: Dada una ronda de mate, computa la lista de las personas ordenadas de menor a mayor según la cantidad de mates que tomaron.

Precondición: Todas las personas de la ronda tomaron al menos un mate.

2. Considerando la siguiente representación:

```
data RondaMate = RM (Ronda Persona) (MultiSet Persona) Int
```

a) Indique invariantes de representación.

b) Implemente las funciones de la interfaz de **RondaMate** respetando las restricciones de eficiencia pedidas. Justifique en cada caso por qué se obtiene la eficiencia buscada.

3. Implemente el TAD **Ronda** eligiendo la representación que considere adecuada, e indicando invariantes de representación y eficiencia de cada función definida.

Interfaces

Interfaz de **Queue**, siendo Q la cantidad de elementos presentes en la cola:

```
emptyQ :: Queue a                O(1)
enqueue :: a -> Queue a -> Queue a  O(1)
firstQ :: Queue a -> a              O(1)
dequeue :: Queue a -> Queue a       O(1)
isEmptyQ :: Queue a -> Bool        O(1)
```

Interfaz de **MultiSet**, siendo M y M' la cantidad de elementos distintos en los multisets:

```
emptyMS :: MultiSet a                O(1)
addMS :: Ord a => a -> MultiSet a -> MultiSet a  O(log M)
ocurrenciasMS :: Ord a => a -> MultiSet a -> Int  O(log M)
unionMS :: Ord a => MultiSet a -> MultiSet a -> MultiSet a  O(M + M')
intersectionMS :: Ord a => MultiSet a -> MultiSet a -> MultiSet a  O(M + M')
multiSetToList :: MultiSet a -> [(Int,a)]        O(M)
```